

PyRaft Lab — The Definitive 15-Day Build

A Raft-based fault-tolerant distributed KV store with deterministic fault injection, automated consistency testing, and performance analytics.

This is the upgraded version — the one that goes from "I implemented Raft" to "I built a distributed systems experimentation platform."

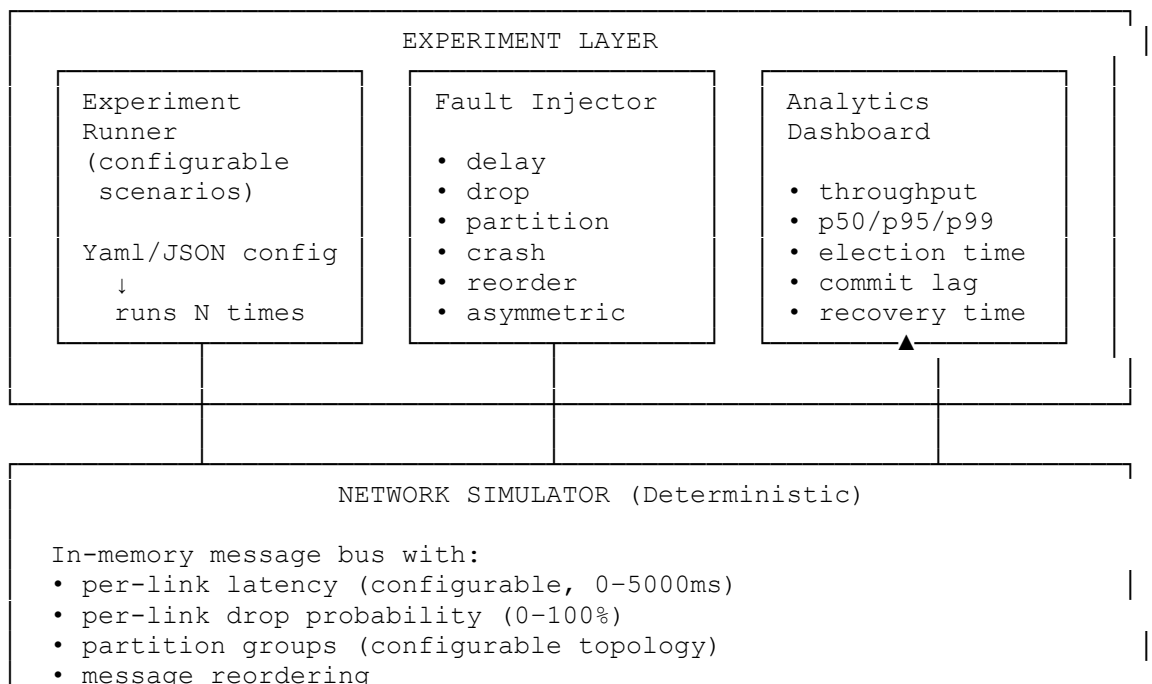
What You're Building (The One-Line Pitch)

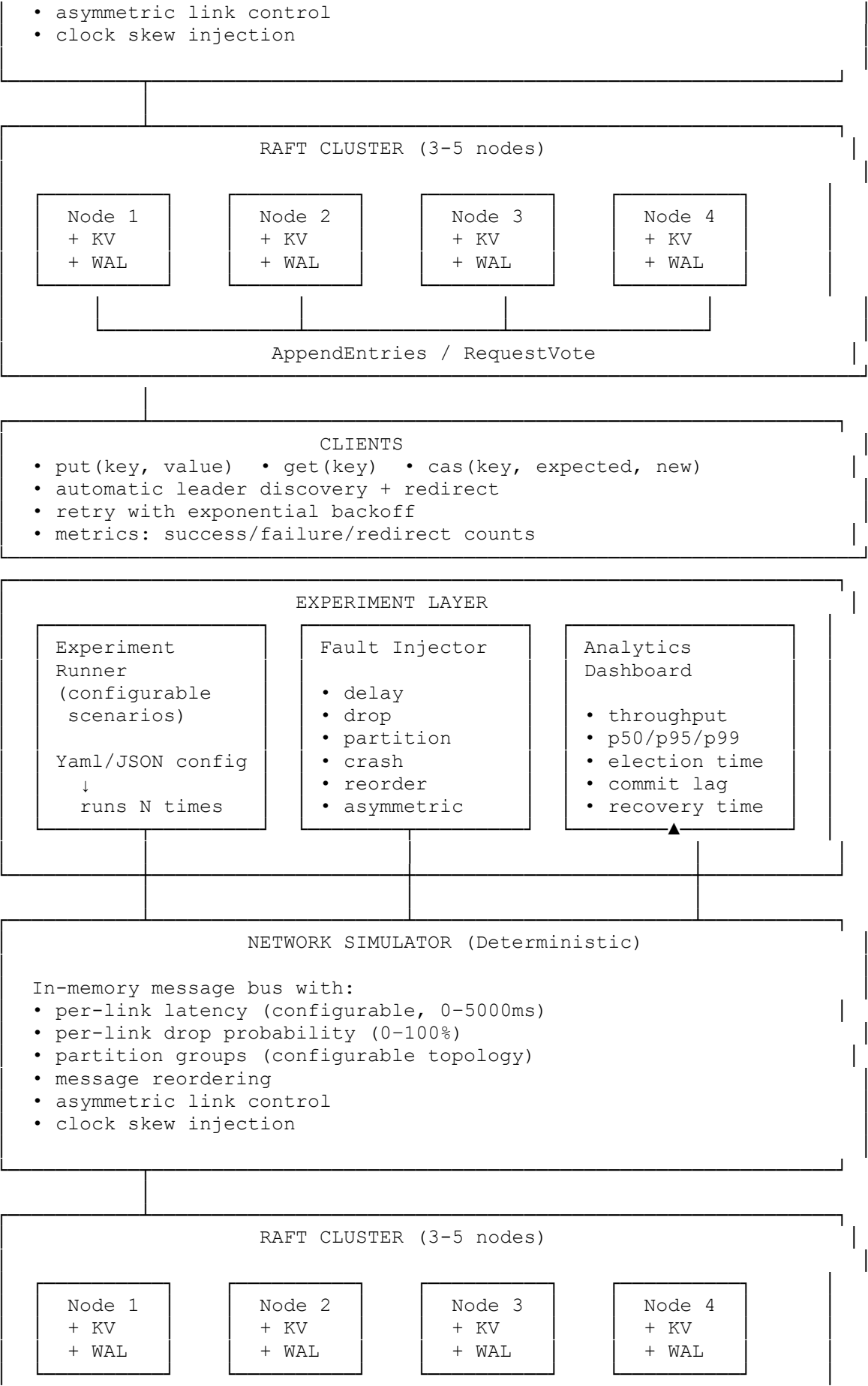
A from-scratch Raft consensus implementation in Python, wrapped around a replicated KV store, with a deterministic fault-injection network simulator and an automated experiment runner that measures election time, commit latency, replication lag, recovery time, and linearizability under various failure modes.

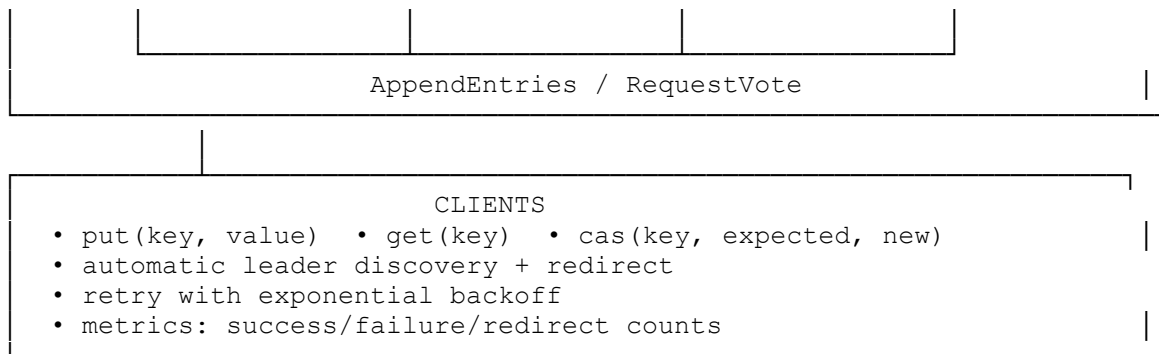
Two layers:

1. The system (Raft + KV)
 2. The laboratory (fault injection + experiments + analytics)
-

Architecture







The Raft Implementation (What Makes It Real)

State Machine (Raft Node)

python

```
class RaftState:
    persistent:
        current_term: int
        voted_for: Optional[str] # candidate_id we voted for in
current term
        log: List[LogEntry] # command + term per entry

    volatile:
        commit_index: int
        last_applied: int
        role: Role # FOLLOWER, CANDIDATE, LEADER

    volatile_leader_only:
        next_index: Dict[NodeId, int] # for each peer, next log entry to
send
        match_index: Dict[NodeId, int] # for each peer, highest
replicated index

    timers:
        election_timeout: Timer # randomized 150-300ms
        heartbeat_interval: Timer # 50ms (for leader)
class RaftState:
    persistent:
        current_term: int
        voted_for: Optional[str] # candidate_id we voted for in
current term
        log: List[LogEntry] # command + term per entry

    volatile:
        commit_index: int
        last_applied: int
        role: Role # FOLLOWER, CANDIDATE, LEADER

    volatile_leader_only:
        next_index: Dict[NodeId, int] # for each peer, next log entry to
send
        match_index: Dict[NodeId, int] # for each peer, highest
replicated index
```

```
timers:
    election_timeout: Timer          # randomized 150-300ms
    heartbeat_interval: Timer       # 50ms (for leader)
```

Five RPCs (implement exactly these)

RPC	Direction	Purpose
RequestVote	Candidate → All	Request vote in election
AppendEntries	Leader → Followers	Replicate log + heartbeat
InstallSnapshot	Leader → Follower (advanced)	Send snapshot when follower is far behind
ClientPut	Client → Leader	Replicate a command
ClientGet	Client → Any	Read with linearizability guarantee

Three Critical Invariants (must hold)

1. Election Safety: At most one leader per term
2. Log Matching: If two logs have an entry with same index+term, all preceding entries are identical
3. State Machine Safety: If a server has applied a log entry at index i, no other server will ever apply a different entry at i



The Fault Injection Layer (Your Differentiator)

This is what separates you from 90% of Raft projects.

Network Simulator Design

python

```
class SimulatedNetwork:
    """Deterministic network with configurable fault injection."""

    def __init__(self, config: NetworkConfig):
        self.topology: Dict[NodeId, Set[NodeId]] # adjacency
        self.link_config: Dict[Tuple[NodeId, NodeId], LinkConfig]
        self.queues: Dict[Link, List[Message]]
        self.clock: VirtualClock

    def send(self, src: NodeId, dst: NodeId, msg: Message):
        """Apply fault injection rules before queuing."""
        link = (src, dst)
        config = self.link_config[link]

        # Drop?
        if random.random() < config.drop_rate:
            self.metrics.dropped += 1
            return # silently dropped

        # Partition?
        if self._is_partitioned(src, dst):
```

```

        self.metrics.partitioned += 1
        return

    # Delay?
    delay = config.latency_ms
    if config.jitter_ms > 0:
        delay += random.uniform(-config.jitter_ms, config.jitter_ms)

    # Reorder? (delay this one to come after next)
    if config.reorder_rate and random.random() < config.reorder_rate:
        delay += config.latency_ms * 2

    # Schedule delivery
    self._schedule_delivery(dst, msg, self.clock.now() + delay)
class SimulatedNetwork:
    """Deterministic network with configurable fault injection."""

    def __init__(self, config: NetworkConfig):
        self.topology: Dict[NodeId, Set[NodeId]] # adjacency
        self.link_config: Dict[Tuple[NodeId, NodeId], LinkConfig]
        self.queues: Dict[Link, List[Message]]
        self.clock: VirtualClock

    def send(self, src: NodeId, dst: NodeId, msg: Message):
        """Apply fault injection rules before queuing."""
        link = (src, dst)
        config = self.link_config[link]

        # Drop?
        if random.random() < config.drop_rate:
            self.metrics.dropped += 1
            return # silently dropped

        # Partition?
        if self._is_partitioned(src, dst):
            self.metrics.partitioned += 1
            return

        # Delay?
        delay = config.latency_ms
        if config.jitter_ms > 0:
            delay += random.uniform(-config.jitter_ms, config.jitter_ms)

        # Reorder? (delay this one to come after next)
        if config.reorder_rate and random.random() < config.reorder_rate:
            delay += config.latency_ms * 2

        # Schedule delivery
        self._schedule_delivery(dst, msg, self.clock.now() + delay)

```

Fault Types You Must Support

Fault	How to Inject	Default Range
Latency	Add delay to all messages	0–500ms
Jitter	Random variance on latency	±50ms
Packet loss	Random drop with probability	0–50%
Network partition	Drop messages between groups	N/A

Fault	How to Inject	Default Range
Node crash	Stop processing for duration	Configurable
Asymmetric link	Different config per direction	Per-link
Message reorder	Reorder queued messages	0–30%
Clock skew	Different node time perception	$\pm N$ seconds
Slow disk	Delay WAL writes	0–100ms

Scenario Configs (YAML)

yaml

```
# scenarios/baseline.yaml
name: "Baseline - No Faults"
duration: 30s
nodes: 3
faults: {}
workload:
  type: "put_get_mix"
  puts_per_sec: 10
  get_ratio: 0.5
expected:
  availability: 1.0
  avg_commit_latency_ms: <50
```

```
# scenarios/leader_crash.yaml
name: "Leader Crash at t=10s"
duration: 30s
nodes: 3
faults:
  - type: node_crash
    target: leader
    start: 10s
    duration: 5s
workload:
  type: "put_get_mix"
  puts_per_sec: 10
expected:
  recovery_time_ms: <3000
  data_loss: false
```

```
# scenarios/minority_partition.yaml
name: "Minority Partition"
duration: 30s
nodes: 5
faults:
  - type: partition
    groups: [[1, 2], [3, 4, 5]]
    start: 5s
    duration: 15s
workload:
  type: "put_get_mix"
  puts_per_sec: 10
expected:
  minority_writes_accepted: 0
```

```

    majority_writes_accepted: true
    no_data_loss: true

---

# scenarios/network_congestion.yaml
name: "High Latency + Packet Loss"
duration: 30s
nodes: 5
faults:
  - type: link_config
    latency_ms: 200
    jitter_ms: 50
    drop_rate: 0.1
    start: 0s
    duration: 30s
workload:
  type: "burst"
  puts_per_sec: 100
# scenarios/baseline.yaml
name: "Baseline - No Faults"
duration: 30s
nodes: 3
faults: {}
workload:
  type: "put_get_mix"
  puts_per_sec: 10
  get_ratio: 0.5
expected:
  availability: 1.0
  avg_commit_latency_ms: <50

---

# scenarios/leader_crash.yaml
name: "Leader Crash at t=10s"
duration: 30s
nodes: 3
faults:
  - type: node_crash
    target: leader
    start: 10s
    duration: 5s
workload:
  type: "put_get_mix"
  puts_per_sec: 10
expected:
  recovery_time_ms: <3000
  data_loss: false

---

# scenarios/minority_partition.yaml
name: "Minority Partition"
duration: 30s
nodes: 5
faults:
  - type: partition
    groups: [[1, 2], [3, 4, 5]]
    start: 5s
    duration: 15s

```

```

workload:
  type: "put_get_mix"
  puts_per_sec: 10
expected:
  minority_writes_accepted: 0
  majority_writes_accepted: true
  no_data_loss: true

---

# scenarios/network_congestion.yaml
name: "High Latency + Packet Loss"
duration: 30s
nodes: 5
faults:
  - type: link_config
    latency_ms: 200
    jitter_ms: 50
    drop_rate: 0.1
    start: 0s
    duration: 30s
workload:
  type: "burst"
  puts_per_sec: 100

```



The Experiment Runner

What It Does

For each scenario YAML:

1. Spawn N Raft nodes + clients
2. Configure the network simulator
3. Run the workload for the duration
4. Collect metrics from every node + client + network
5. Compute summary statistics
6. Write results to JSON + plots

Result Schema

```

json
{
  "scenario": "leader_crash.yaml",
  "run_id": "abc123",
  "duration_sec": 30.0,
  "summary": {
    "total_requests": 287,
    "successful": 285,
    "failed_redirects": 2,
    "availability": 0.993,
    "data_loss": false
  },
  "timing": {
    "avg_commit_latency_ms": 23.4,
    "p50_commit_latency_ms": 18.2,

```



```

    "p95_commit_latency_ms": 67.1,
    "p99_commit_latency_ms": 124.5
  },
  "leadership": {
    "total_elections": 1,
    "election_time_ms": 1247,
    "leader_changes": 1,
    "leadership_timeline": [
      {"term": 1, "leader": "node1", "start": 0, "end": 10247},
      {"term": 2, "leader": "node2", "start": 10247, "end": 30000}
    ]
  },
  "fault_impact": {
    "messages_sent": 4521,
    "messages_delivered": 4102,
    "messages_dropped": 419,
    "messages_delayed": 1247
  }
}
{
  "scenario": "leader_crash.yaml",
  "run_id": "abc123",
  "duration_sec": 30.0,
  "summary": {
    "total_requests": 287,
    "successful": 285,
    "failed_redirects": 2,
    "availability": 0.993,
    "data_loss": false
  },
  "timing": {
    "avg_commit_latency_ms": 23.4,
    "p50_commit_latency_ms": 18.2,
    "p95_commit_latency_ms": 67.1,
    "p99_commit_latency_ms": 124.5
  },
  "leadership": {
    "total_elections": 1,
    "election_time_ms": 1247,
    "leader_changes": 1,
    "leadership_timeline": [
      {"term": 1, "leader": "node1", "start": 0, "end": 10247},
      {"term": 2, "leader": "node2", "start": 10247, "end": 30000}
    ]
  },
  "fault_impact": {
    "messages_sent": 4521,
    "messages_delivered": 4102,
    "messages_dropped": 419,
    "messages_delayed": 1247
  }
}

```



Detailed 15-Day Plan

Phase 1: Foundation (Days 1-6)

Both people work together on foundational pieces

Day 1: Project Setup + RPC Layer

- Repo structure, pyproject.toml, CI skeleton
- Decide RPC: pure `asyncio` streams vs `grpcio` (recommend: `asyncio` for simplicity, `gRPC` is overkill)
- Implement message serialization (JSON or `msgpack`)
- Basic in-memory network bus (no faults yet)
- Deliverable: Two nodes can send/receive arbitrary messages

Day 2: Raft State Machine + RPCs

- `RaftState` class with all persistent + volatile state
- `RequestVote` RPC (args, reply, handler)
- `AppendEntries` RPC
- State transition logic (Follower ↔ Candidate ↔ Leader)
- Deliverable: Static 3-node cluster, all in `Follower` state

Day 3: Leader Election

- Election timer (randomized 150-300ms)
- Become candidate, vote for self
- Send `RequestVote` to all peers
- Tally votes, become leader on majority
- Test: Start 3 nodes, watch one become leader via logs
- Test: Kill leader, new election within 300ms

Day 4: Log Replication

- Leader appends client commands to its log
- Send `AppendEntries` to all followers
- Followers reject if `prevLogTerm/prevLogIndex` don't match
- Leader decrements `nextIndex` on rejection, retries
- Commit when replicated to majority
- Test: Put 100 commands, verify all nodes have same log

Day 5: KV State Machine + Apply

- State machine applies committed log entries
- KV operations: `put`, `get`, `delete`, `cas` (compare-and-swap)
- Last-applied tracking
- Test: `put foo bar` on leader, `get foo` returns `bar` from all nodes

Day 6: Client Library + Leader Discovery

- Client knows cluster topology (config file)
- Client sends to any node; redirects to leader on `NOT_LEADER`
- Client retries on failure with backoff
- Test: Client does 1000 ops with no failures under normal conditions

- WEEK 1 MILESTONE: 3-node Raft KV working, no faults yet
-

Phase 2: Failure Handling (Days 7-10)

Day 7: Failure Detection + Recovery

- Heartbeat mechanism (leader sends empty `AppendEntries` every 50ms)
- Follower resets election timer on valid `AppendEntries`
- Stale leader steps down on higher term in response
- Test: Kill leader, recovery in <2s
- Test: Old leader comes back, recognizes new leader, steps down

Day 8: Network Simulator (Basic)

- Replace direct message passing with `SimulatedNetwork`
- Per-link latency configuration
- Per-link drop rate
- Test: 200ms latency, all ops still succeed, just slower

Day 9: Network Partitions

- Partition group configuration
- Drop messages between groups
- Minority side: elections fail (no quorum)
- Majority side: continues, may elect new leader
- Test: Partition 1 node, it can't elect leader; other 2 continue
- Test: Heal partition, stale node catches up

Day 10: Linearizable Reads (Read-Index or Lease)

- Implement read-index: leader confirms it's still leader before serving read
 - Or: leader lease (assumes no other leader for election timeout duration)
 - Test: During partition, old leader rejects reads
 - WEEK 2 START: Fault injection working, linearizability addressed
-

Phase 3: The Laboratory (Days 11-13)

Day 11: Experiment Runner + Scenario Configs

- YAML scenario parser
- Run N trials per scenario, aggregate results
- Per-node metrics collection (events: state change, vote, replicate, commit, apply)
- Per-client metrics (request, success, redirect, latency)
- Test: Run "baseline" and "leader_crash" scenarios, get JSON output

Day 12: Metrics + Visualization

- Compute: throughput, p50/p95/p99 latency, election time, recovery time, availability
- matplotlib plots: latency over time, leadership timeline, commit lag heatmap
- Save plots to `results/{scenario}/plots/`
- Test: Generate comparison plot: "leader crash recovery across 10 trials"

Day 13: Linearizability Verification

- Use a simple concurrent history checker
- Record all client operations with timestamps + values
- Verify the history is linearizable (or document near-misses)
- Test: Under partition, verify no stale reads from old leader

Phase 4: Polish + Ship (Days 14-15)

Day 14: Documentation + Benchmarks

- README with quickstart (`pip install, python -m pyraft_lab run --scenario baseline`)
- Architecture diagram (Mermaid or PlantUML)
- 4-page technical report (design decisions, fault tolerance analysis, lessons)
- Run final benchmark suite: 10+ scenarios, generate report

Day 15: Demo + Release

- Record 5-min demo: baseline → leader crash → partition → recovery → analytics
- Final integration test
- Tag `v0.1.0`, GitHub release
- Buffer for last-minute bugs

Project Structure

```
pyraft_lab/
├── src/pyraft_lab/
│   ├── raft/
│   │   ├── __init__.py
│   │   ├── node.py           # RaftNode class
│   │   ├── state.py         # State machine + persistent state
│   │   ├── rpc.py           # RequestVote, AppendEntries handlers
│   │   ├── log.py           # Log entry + storage
│   │   ├── election.py      # Election logic
│   │   ├── replication.py   # Log replication logic
│   │   └── persistence.py   # WAL (write-ahead log)
│   ├── kv/
│   │   ├── __init__.py
│   │   ├── statemachine.py  # Applies committed entries to KV
│   │   └── operations.py    # put/get/delete/cas
│   └── network/
│       └── __init__.py
```

```

├── simulator.py          # SimulatedNetwork with faults
├── link.py               # Per-link configuration
├── clock.py             # Virtual clock for determinism
├── client/
│   ├── __init__.py
│   ├── client.py        # Client with leader discovery
│   └── workload.py      # Workload generators
├── lab/
│   ├── __init__.py
│   ├── runner.py        # Experiment runner
│   ├── scenario.py      # YAML scenario loader
│   ├── metrics.py       # Metrics collection + aggregation
│   └── visualization.py  # Plot generation
├── cli/
│   └── main.py          # `pyraft-lab` CLI entry point
├── scenarios/
│   ├── baseline.yaml
│   ├── leader_crash.yaml
│   ├── minority_partition.yaml
│   ├── majority_partition.yaml
│   ├── high_latency.yaml
│   ├── packet_loss.yaml
│   └── network_congestion.yaml
├── tests/
│   ├── test_election.py
│   ├── test_replication.py
│   ├── test_persistence.py
│   ├── test_fault_injection.py
│   └── test_linearizability.py
├── results/              # Generated by experiments
│   └── {scenario}/
│       ├── results.json
│       ├── timeline.png
│       ├── latency.png
│       └── leadership.png
├── docs/
│   ├── architecture.md
│   ├── report.pdf
│   └── demo.mp4
├── pyproject.toml
├── README.md
├── LICENSE
pyraft_lab/
├── src/pyraft_lab/
│   ├── raft/
│   │   ├── __init__.py
│   │   ├── node.py      # RaftNode class
│   │   ├── state.py     # State machine + persistent state
│   │   ├── rpc.py       # RequestVote, AppendEntries handlers
│   │   ├── log.py       # Log entry + storage
│   │   ├── election.py   # Election logic
│   │   ├── replication.py # Log replication logic
│   │   └── persistence.py # WAL (write-ahead log)
│   ├── kv/
│   │   ├── __init__.py
│   │   ├── statemachine.py # Applies committed entries to KV
│   │   └── operations.py  # put/get/delete/cas
│   └── network/
│       ├── __init__.py
│       ├── simulator.py  # SimulatedNetwork with faults
│       └── link.py       # Per-link configuration

```

```

├── clock.py                # Virtual clock for determinism
├── client/
│   ├── __init__.py
│   ├── client.py          # Client with leader discovery
│   └── workload.py        # Workload generators
├── lab/
│   ├── __init__.py
│   ├── runner.py          # Experiment runner
│   ├── scenario.py        # YAML scenario loader
│   ├── metrics.py         # Metrics collection + aggregation
│   └── visualization.py   # Plot generation
├── cli/
│   └── main.py            # `pyraft-lab` CLI entry point
├── scenarios/
│   ├── baseline.yaml
│   ├── leader_crash.yaml
│   ├── minority_partition.yaml
│   ├── majority_partition.yaml
│   ├── high_latency.yaml
│   ├── packet_loss.yaml
│   └── network_congestion.yaml
├── tests/
│   ├── test_election.py
│   ├── test_replication.py
│   ├── test_persistence.py
│   ├── test_fault_injection.py
│   └── test_linearizability.py
├── results/                # Generated by experiments
│   └── {scenario}/
│       ├── results.json
│       ├── timeline.png
│       ├── latency.png
│       └── leadership.png
├── docs/
│   ├── architecture.md
│   ├── report.pdf
│   └── demo.mp4
├── pyproject.toml
├── README.md
└── LICENSE

```

Tech Stack (Concrete)

Layer	Choice	Rationale
Language	Python 3.11+	Type hints, async, clean
Async	asyncio	Built-in, no deps, perfect for I/O simulation
Serialization	msgpack or pydantic + JSON	Fast, self-describing
Persistence	aiosqlite or append-only JSON files	Simple WAL
Config	pydantic + YAML	Type-safe scenario configs
CLI	typer or click	Modern CLI framework
Testing	pytest + pytest-asyncio	Standard

Layer	Choice	Rationale
Metrics	Custom in-memory + prometheus_client	For experiment aggregation
Visualization	matplotlib + plotly	Static + interactive
Linearizability check	Custom (small) or knossos lib	Validate histories
CI	GitHub Actions	Lint, test, fuzz

Total external dependencies: ~10-12 (very manageable)

The Killer Demo (5 minutes)

```
[0:00] Title: "PyRaft Lab – Distributed Consensus Under Failure"
[0:15] Architecture overview (1 min)
[1:15] Live demo:
  - 3 nodes boot
  - 100 puts succeed, all replicated
  - `pyraft-lab show log` shows consistent state
[2:30] Experiment 1: Leader crash
  - `pyraft-lab run --scenario leader_crash`
  - Logs show: leader dies → election in 1.2s → new leader
  - No data loss verified
[3:15] Experiment 2: Minority partition
  - `pyraft-lab run --scenario minority_partition`
  - Minority side: writes rejected (no quorum)
  - Majority side: continues serving
[3:45] Experiment 3: Network chaos
  - 200ms latency + 10% packet loss
  - System still works, just slower
[4:00] Analytics dashboard
  - Latency percentiles across scenarios
  - Recovery time distribution
  - Leadership timeline visualization
[4:30] Linearizability check
  - Concurrent workload history verified
[4:45] Code stats: ~3000 LOC, 40+ tests, 7 scenarios
[5:00] Closing: GitHub link, "Built in 15 days"
[0:00] Title: "PyRaft Lab – Distributed Consensus Under Failure"
[0:15] Architecture overview (1 min)
[1:15] Live demo:
  - 3 nodes boot
  - 100 puts succeed, all replicated
  - `pyraft-lab show log` shows consistent state
[2:30] Experiment 1: Leader crash
  - `pyraft-lab run --scenario leader_crash`
  - Logs show: leader dies → election in 1.2s → new leader
  - No data loss verified
[3:15] Experiment 2: Minority partition
  - `pyraft-lab run --scenario minority_partition`
  - Minority side: writes rejected (no quorum)
  - Majority side: continues serving
[3:45] Experiment 3: Network chaos
  - 200ms latency + 10% packet loss
  - System still works, just slower
```

```
[4:00] Analytics dashboard
- Latency percentiles across scenarios
- Recovery time distribution
- Leadership timeline visualization
[4:30] Linearizability check
- Concurrent workload history verified
[4:45] Code stats: ~3000 LOC, 40+ tests, 7 scenarios
[5:00] Closing: GitHub link, "Built in 15 days"
```

Must-Have Scenarios (Minimum Viable Lab)

Ship at least these 7 scenarios:

#	Scenario	What It Tests
1	baseline	No faults, measure normal performance
2	leader_crash	Recovery from leader failure
3	minority_partition	Quorum rejection
4	majority_partition	Both sides try to elect, only majority succeeds
5	network_latency	High latency tolerance
6	packet_loss	Lossy network handling
7	churn	Continuous node restarts

Bonus scenarios (if time):

- asymmetric_link
 - clock_skew
 - message_reorder
 - disk_pause
 - cascading_failures
-

Required Metrics (Per Experiment)

Metric	How to Measure	Why It Matters
Availability	(successful_requests / total_requests)	CAP theorem in action
Avg commit latency	(commit_time - client_send_time)	Performance
p50/p95/p99 latency	Percentile of commit times	Tail behavior
Election time	(new_leader_elected - old_leader_dead)	Failover speed
Recovery time	(first_commit_after_fault - fault_start)	Resilience
Replication lag	(follower_match_index - leader_commit) per node	Consistency
Election count	Number of elections during run	Stability

Metric	How to Measure	Why It Matters
Data loss	(committed_but_not_in_new_leader)	Safety property
Stale reads	Reads served after leader lost leadership	Linearizability

Testing Strategy

Unit Tests (Days 1-6)

- Election: candidate becomes leader on majority
- Election: split vote, retry with new term
- Replication: leader replicates to all followers
- Replication: follower rejects mismatched entries
- Commit: entry committed only when on majority
- Apply: state machine applies committed entries in order

Integration Tests (Days 7-10)

- Leader crash → new leader elected, no data loss
- Minority partition → no writes accepted
- Majority partition → continues, may elect new leader
- Partition heal → stale followers catch up
- Network latency → all ops eventually succeed
- Packet loss → retry logic works

Property Tests (Day 11)

- Safety: No two committed entries at same index with different terms
 - Safety: No data loss across N runs of leader_crash scenario
 - Liveness: System makes progress when majority is available
 - Linearizability: All client histories are linearizable
-

Critical Risk Areas

Risk	Mitigation
Election loops (split votes forever)	Randomized timeout, ensure > 2 candidates can resolve
Log divergence after partition	Strict <code>prevLogTerm/prevLogIndex</code> checks, log truncation on conflict
Stale leader serving reads	Implement read-index or lease for linearizable reads
Persistence corruption	CRC on WAL, <code>fsync</code> before responding to RPC
Python async race conditions	Single-threaded per node, use <code>asyncio.Lock</code> only when needed

Risk	Mitigation
Test flakiness	Deterministic network sim (no real network), use virtual clock

The "9.3 → 9.5" Differentiators

If you want to push from 9.3 to 9.5, add these (pick 1-2):

1. Raft log visualization — `manim` or `matplotlib` animation showing the log across nodes evolving
 2. Comparison with simple replication — Run the same workload against non-Raft "last-writer-wins" replication, show Raft's linearizability wins
 3. Failure matrix benchmark — Run all 7 scenarios × 10 trials, generate a heatmap of availability vs fault type
 4. TLA+ spec alongside Python — Write a TLA+ spec of your protocol, model-check it, then implement in Python. This is a huge flex.
 5. Read-the-docs site with auto-generated API docs and architecture diagrams
-